Assignment Report

SFWRENG 3S03 (2026W)

Assignment 2

Author(s)

Salma Baig baigs19@mcmaster.ca baigs19

GitHub URL

`https://github.com/hydrowoxy/3S03-A2`

March 27, 2026

Original template credits: Samuel Zurowski and Charles Barone. Adapted for McMaster SFWRENG by Dr. Istvan David.

1 Basic Deployment Pipeline - Modeling with Petri Nets

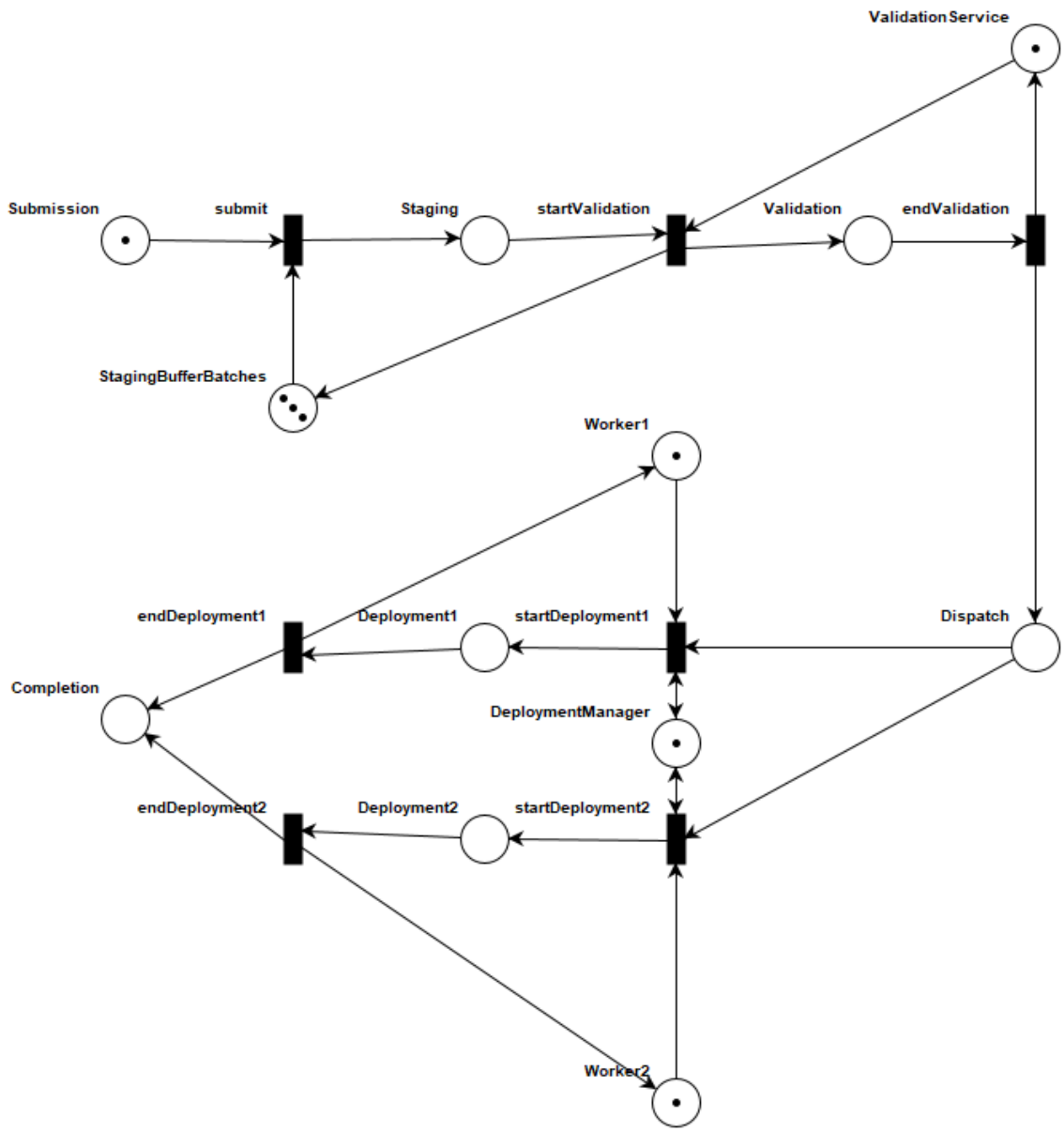


Figure 1: Petri net model of the deployment pipeline.

In the Petri net above, tokens represent either deployment batches or resource availability, depending on the place. Tokens in pipeline places represent deployment batches progressing through the system, while tokens in resource places represent the availability of shared resources.

The places in the model are as follows:

- Submission: contains deployment batches that have entered the system and are waiting

to be staged.

- Staging: contains batches currently waiting in the staging buffer.
- Validation: contains a batch that is currently being validated.
- Dispatch: contains a validated batch that is ready to be assigned to a deployment worker.
- Deployment1: contains a batch currently being deployed by worker 1.
- Deployment2: contains a batch currently being deployed by worker 2.
- Completion: contains batches that have finished deployment successfully.
- StagingBufferBatches: contains tokens representing free capacity in the staging buffer. Since it initially contains three tokens, at most three batches can be in Staging at once.
- ValidationService: contains a token representing that the validation service is available. Since it contains one token, only one batch can be validated at a time.
- DeploymentManager: contains a token representing that the deployment manager is available. Since it contains one token, only one dispatch decision can be made at a time.
- Worker1: contains a token representing that deployment worker 1 is available.
- Worker2: contains a token representing that deployment worker 2 is available.

The transitions in the model are as follows:

- submit: moves a batch from Submission to Staging. Consumes one token from StagingBufferBatches, so a staging slot is used.
- startValidation: moves a batch from Staging to Validation. Uses the ValidationService token and returns the StagingBufferBatches token, freeing a staging slot.
- endValidation: moves a batch from Validation to Dispatch and returns the ValidationService token, making the validation service available again.
- startDeployment1: moves a batch from Dispatch to Deployment1. Uses both the DeploymentManager token and the Worker1 token. The DeploymentManager token is returned immediately, since the manager is only needed to assign the batch, while Worker1 remains unavailable until the deployment completes.
- endDeployment1: moves a batch from Deployment1 to Completion and returns the Worker1 token, making worker 1 available again.
- startDeployment2: Same as 1, but for worker 2.
- endDeployment2: Same as 1, but for worker 2.

2 Safety and other useful properties - Formalizing requirements

2.1 Buffer Safety

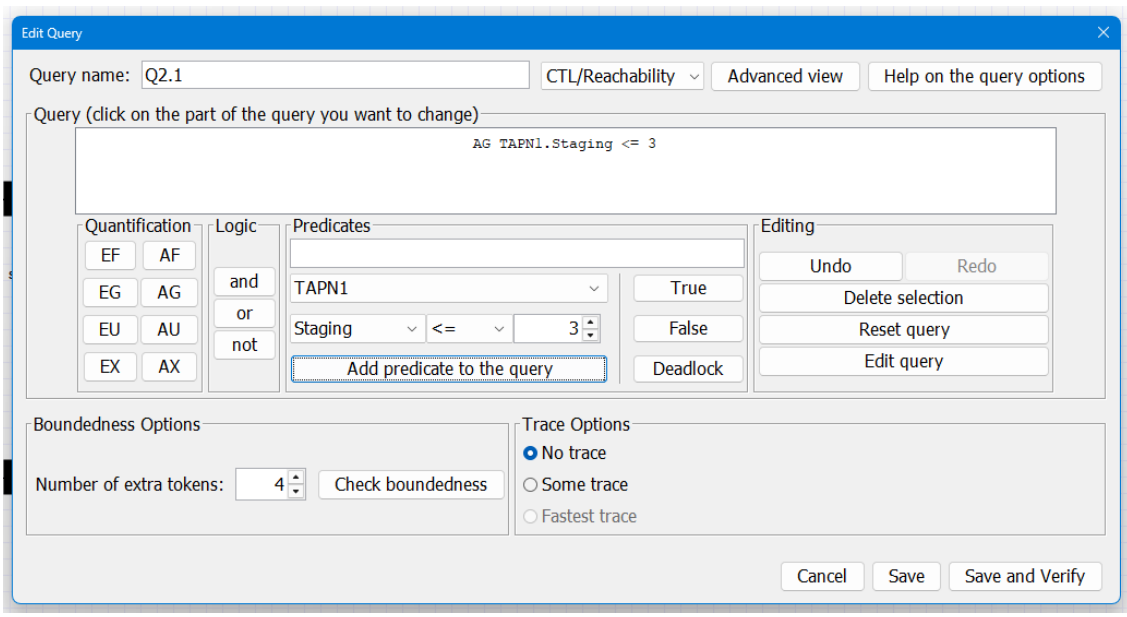
i.

$AG(Staging \leq 3)$

This query states that on all execution paths, the number of tokens in the Staging place is always at most 3. This ensures the staging buffer capacity is never exceeded.

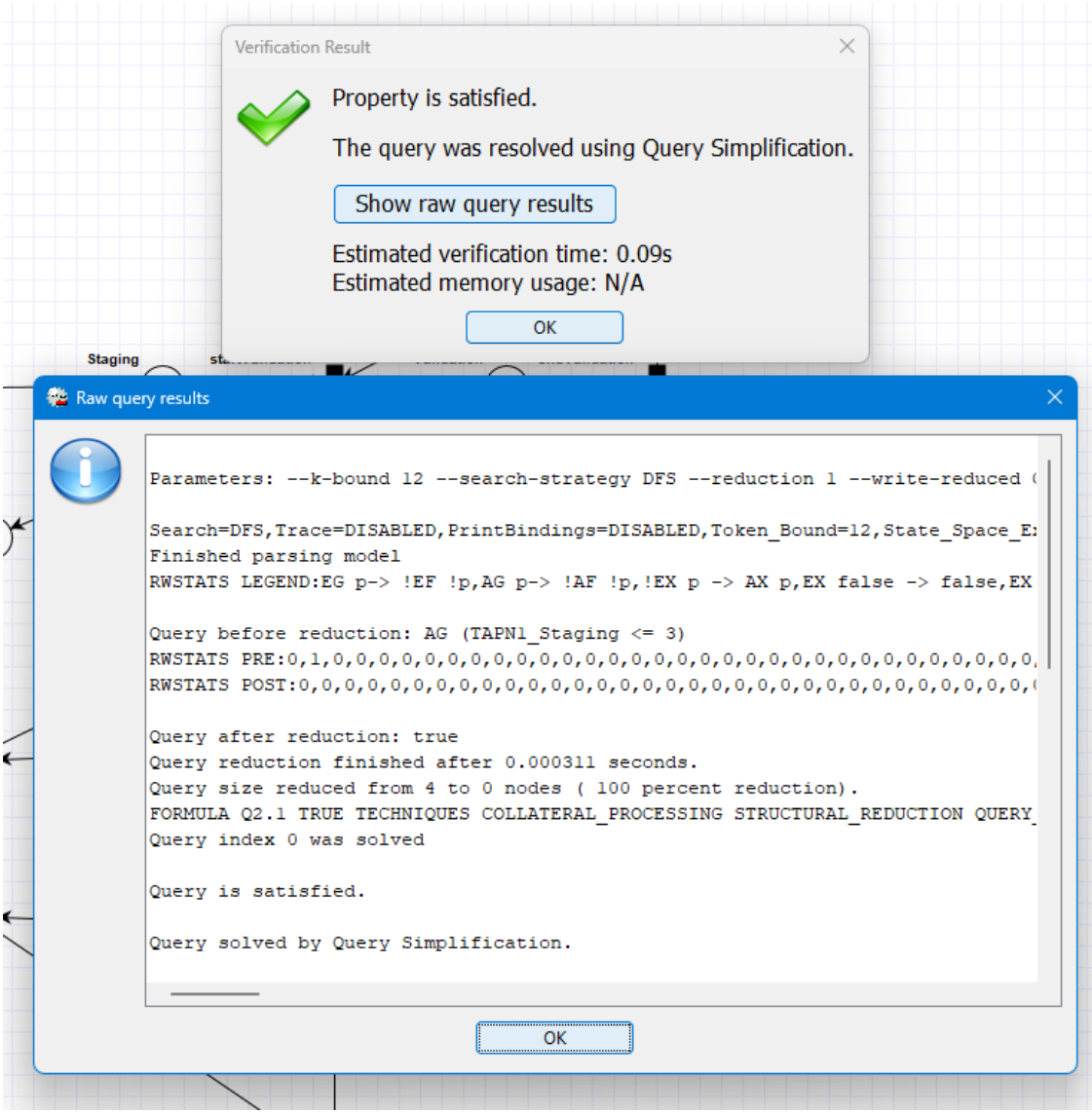
ii.

```
AG (Staging <= 3)
```



iii.

TAPAAL evaluates this query as TRUE. This means that in all reachable markings, the number of tokens in Staging never exceeds three, so the model correctly enforces the buffer capacity constraint.



2.2 Resource Exclusivity

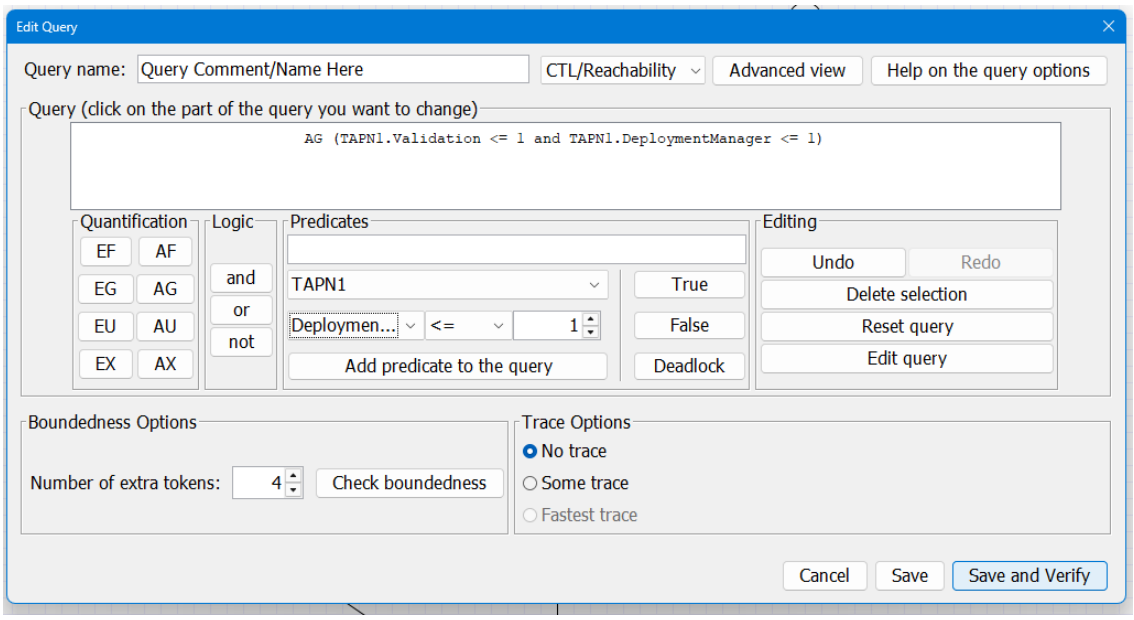
i.

$AG(\text{Validation} \leq 1 \wedge \text{DeploymentManager} \leq 1)$

This query states that on all execution paths, the number of tokens in Validation and DeploymentManager never exceeds 1. This ensures that the validation service and deployment manager are never used by more than one batch at the same time.

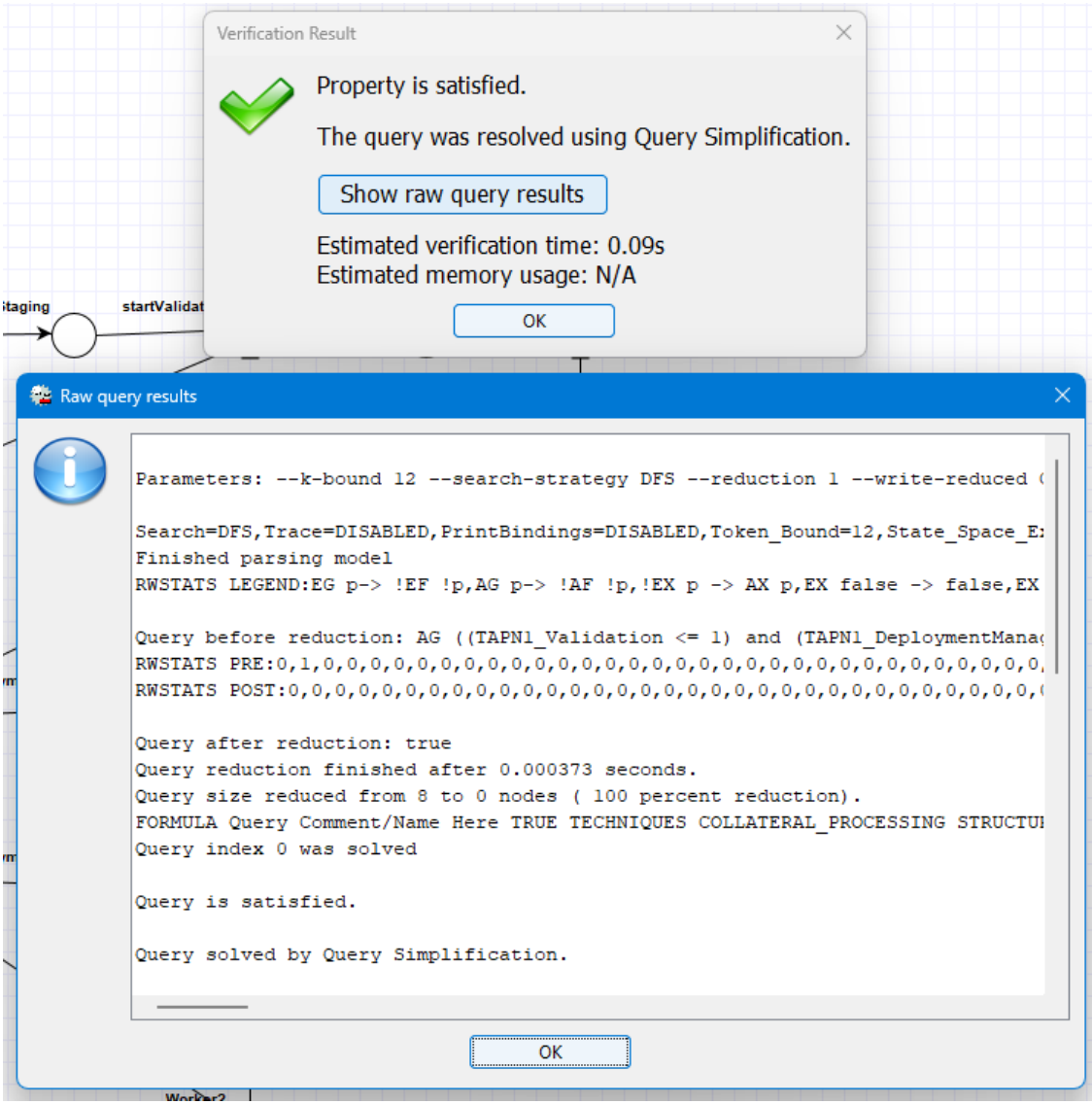
ii.

```
AG (Validation <= 1 and DeploymentManager <= 1)
```



iii.

TAPAAL evaluates this query as TRUE. This means that, in all reachable markings, neither the validation service nor the deployment manager is used by more than one batch simultaneously, so the model correctly enforces mutual exclusion.



2.3 Deadlock Possibility

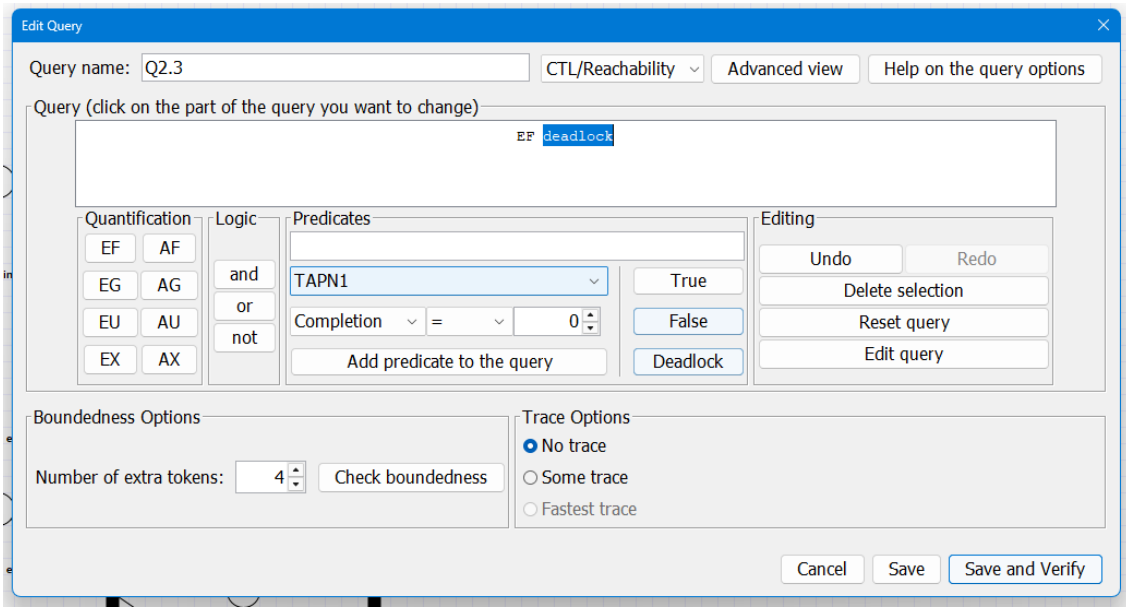
i.

$EF(\text{deadlock})$

This query checks whether there exists a reachable marking in which no transitions are enabled, indicating a deadlock.

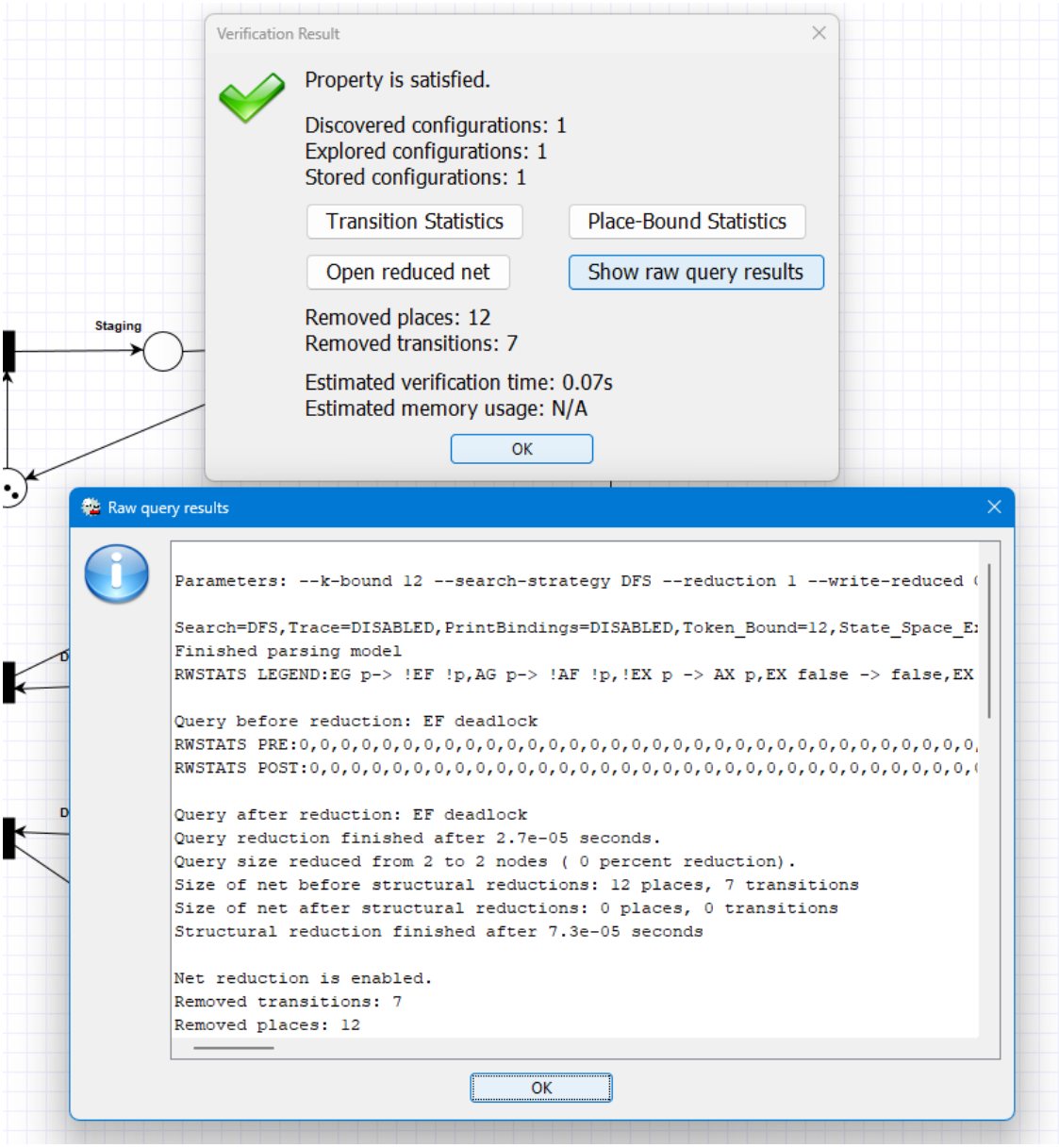
ii.

`EF deadlock`



iii.

TAPAAL evaluates this query as TRUE. This means that there exists a reachable marking in which no transitions are enabled, so the system can deadlock.



2.4 Pipeline Progress

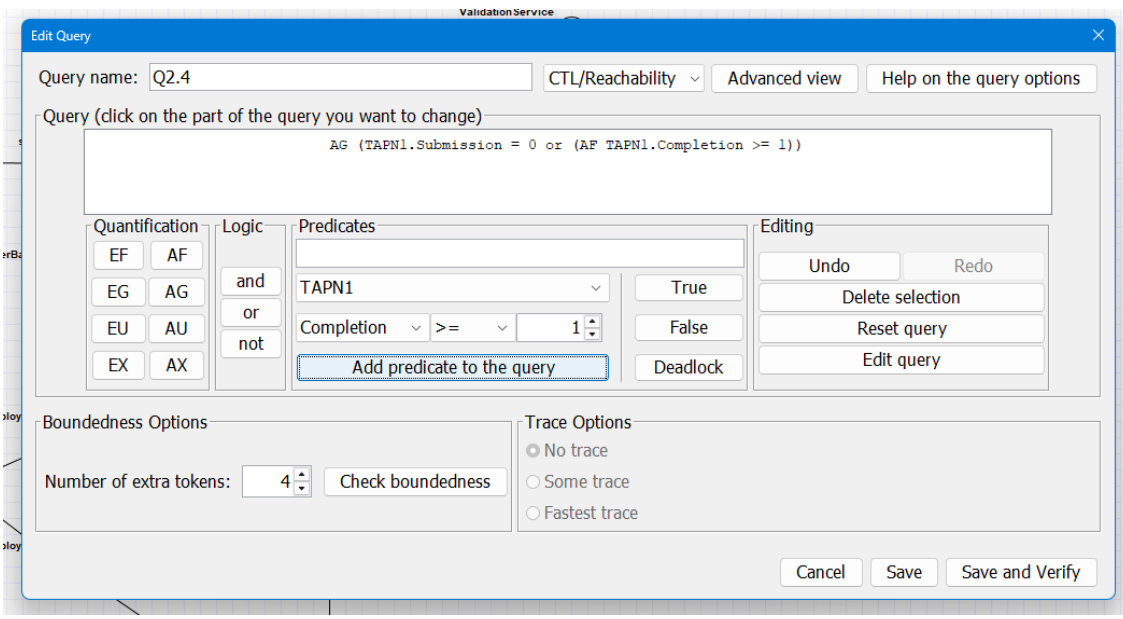
i.

$AG(Submission = 0 \vee AF(Completion \geq 1))$

This query states that in every reachable state, either there is no batch remaining in Submission, or a batch will eventually reach Completion. This ensures that whenever a batch is present in Submission, eventually some batch reaches Completion.

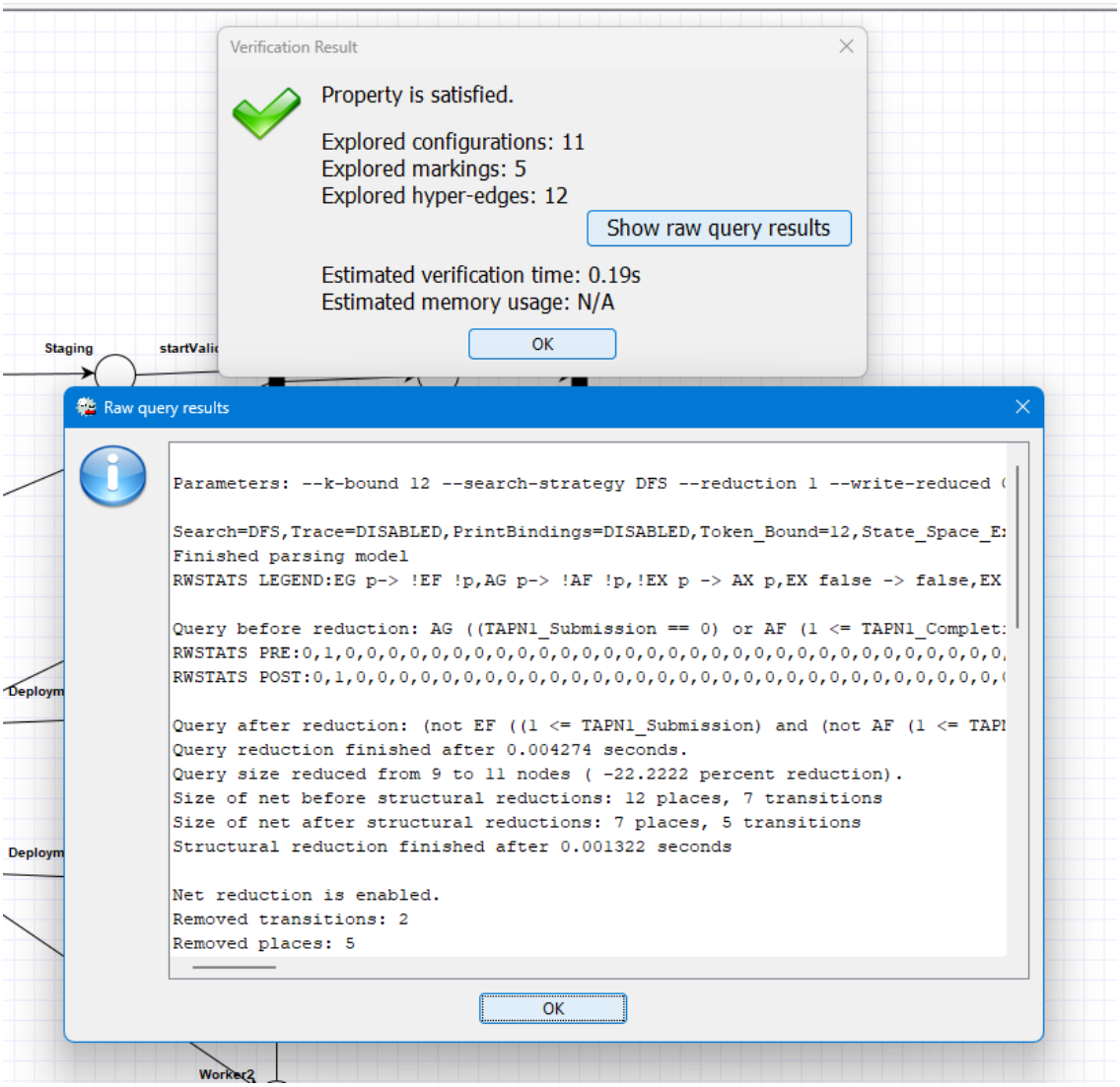
ii.

```
AG (Submission == 0 or AF (Completion >= 1))
```



iii.

TAPAAL evaluates this query as TRUE. This means that whenever a batch is still present at Submission, the model guarantees that eventually some batch will reach Completion.


Verification Result
Property is satisfied.
Explored configurations: 11
Explored markings: 5
Explored hyper-edges: 12
Show raw query results
Estimated verification time: 0.19s
Estimated memory usage: N/A
OK
Staging
Raw query results
Parameters: --k-bound 12 --search-strategy DFS --reduction 1 --write-reduced (
Search=DFS,Trace=DISABLED,PrintBindings=DISABLED,Token_Bound=12,State_Space_E:
Finished parsing model
RWSTATS LEGEND:EG p-> !EF !p,AG p-> !AF !p,!EX p -> AX p,EX false -> false,EX
Query before reduction: AG ((TAPN1_Submission == 0) or AF (1 <= TAPN1_Complet:
RWSTATS PRE:0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
RWSTATS POST:0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
Query after reduction: (not EF ((1 <= TAPN1_Submission) and (not AF (1 <= TAP
Query reduction finished after 0.004274 seconds.
Query size reduced from 9 to 11 nodes (-22.2222 percent reduction).
Size of net before structural reductions: 12 places, 7 transitions
Size of net after structural reductions: 7 places, 5 transitions
Structural reduction finished after 0.001322 seconds
Net reduction is enabled.
Removed transitions: 2
Removed places: 5
OK
Deploym
Deploym
Worker2

3 Critical updates - Working with priorities

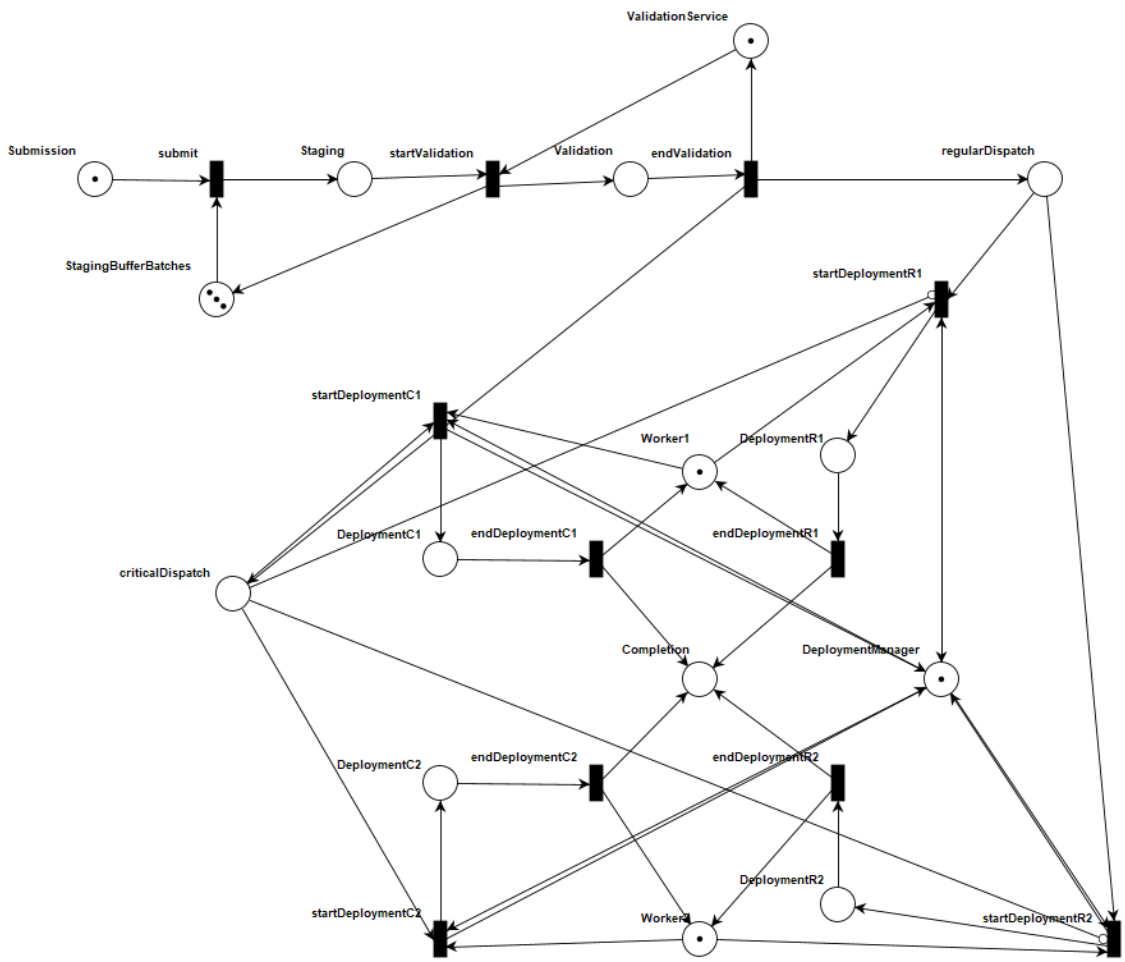


Figure 2: Petri net model of the extended deployment pipeline.

The Petri net from Q1 was extended in the following ways:

- The original single waiting place was split into two distinct places representing regular and critical updates.
- The original assignment transition was replaced with two transitions: one for assigning regular updates and one for assigning critical updates.
- An inhibitor arc was added from the critical update waiting place to the regular assignment transition. This disables the assignment of regular updates whenever at least one critical update is present, enforcing the priority policy.
- DeploymentManager manages both regular and critical updates.

4 Formalizing Requirements

4.1 Priority Enforcement

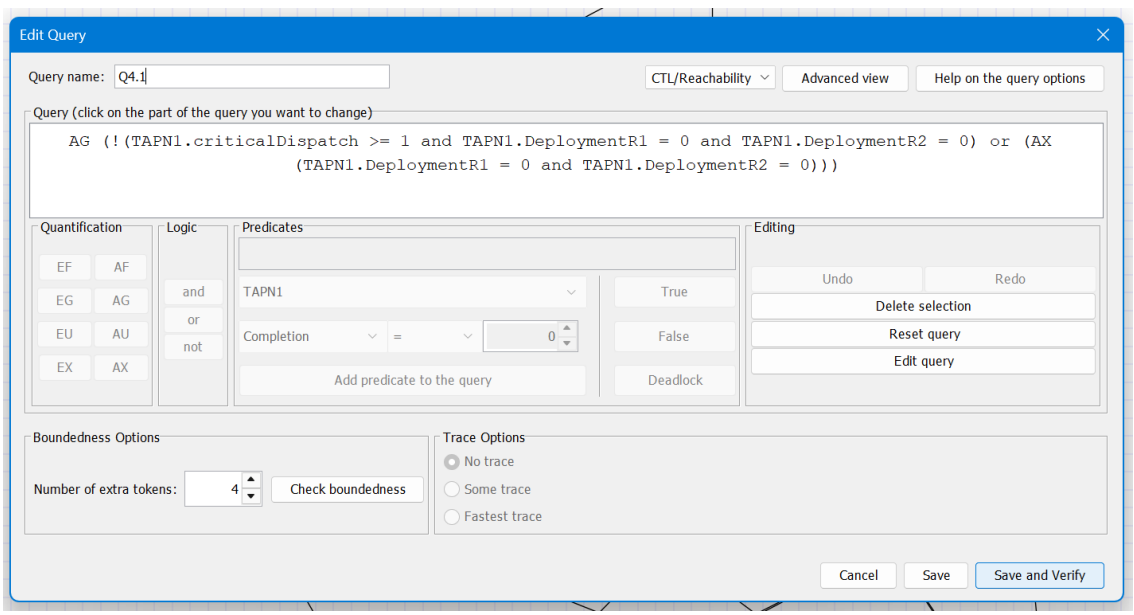
i.

$AG(\neg(criticalDispatch \geq 1 \wedge DeploymentR1 = 0 \wedge DeploymentR2 = 0) \vee AX(DeploymentR1 = 0 \wedge DeploymentR2 = 0))$

This query states that whenever a critical update is waiting to be dispatched and no regular deployment is currently active, the next state cannot start deployment of a regular update. So, regular updates cannot begin deployment while a critical update is waiting.

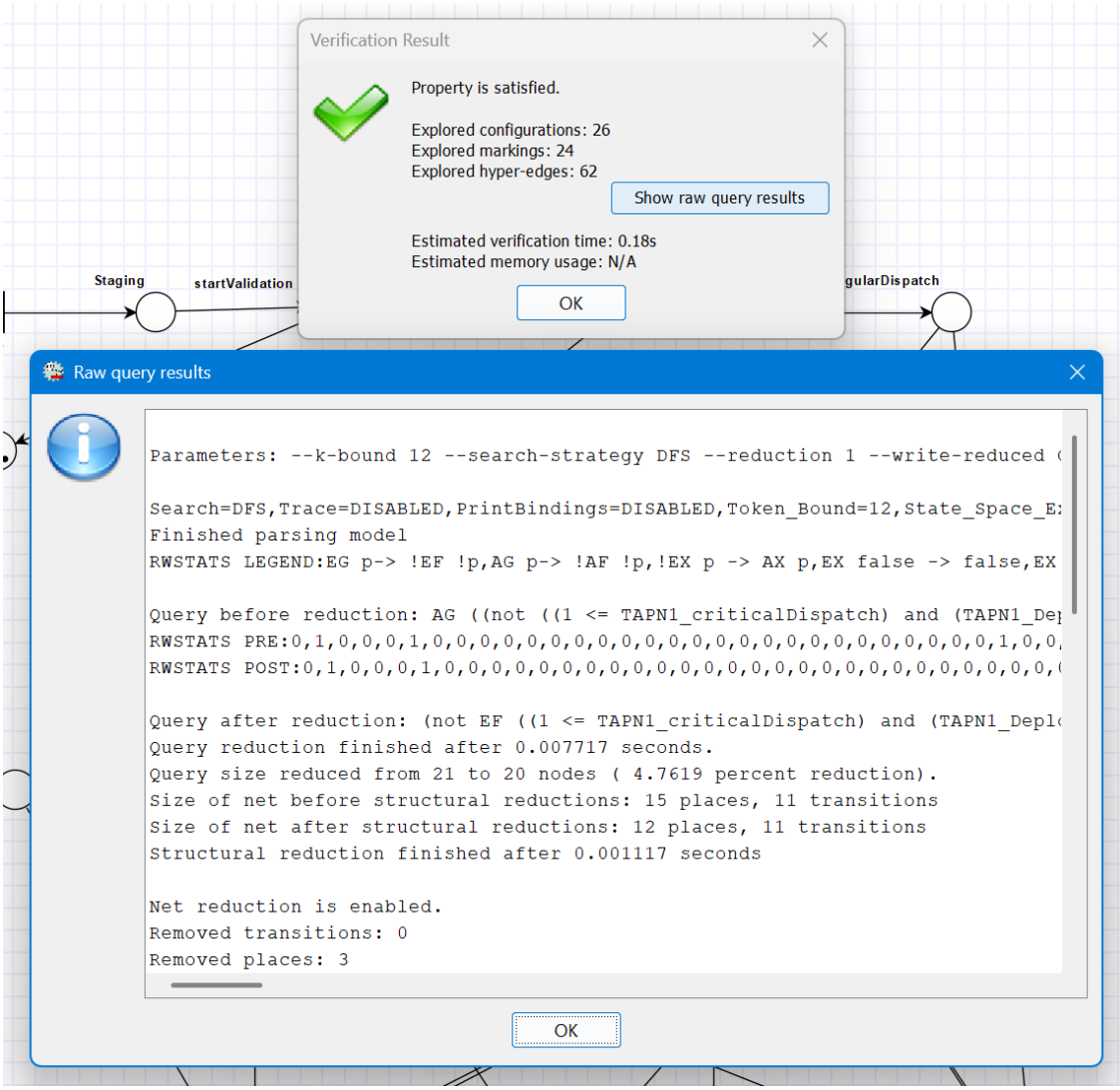
ii.

```
AG (!(criticalDispatch >= 1 and DeploymentR1 == 0 and DeploymentR2 == 0) or AX
(DeploymentR1 == 0 and DeploymentR2 == 0))
```



iii.

TAPAAL evaluates this query as TRUE. This means that the model correctly enforces priority by preventing regular deployments from starting whenever a critical update is waiting.



4.2 Critical Update Progress

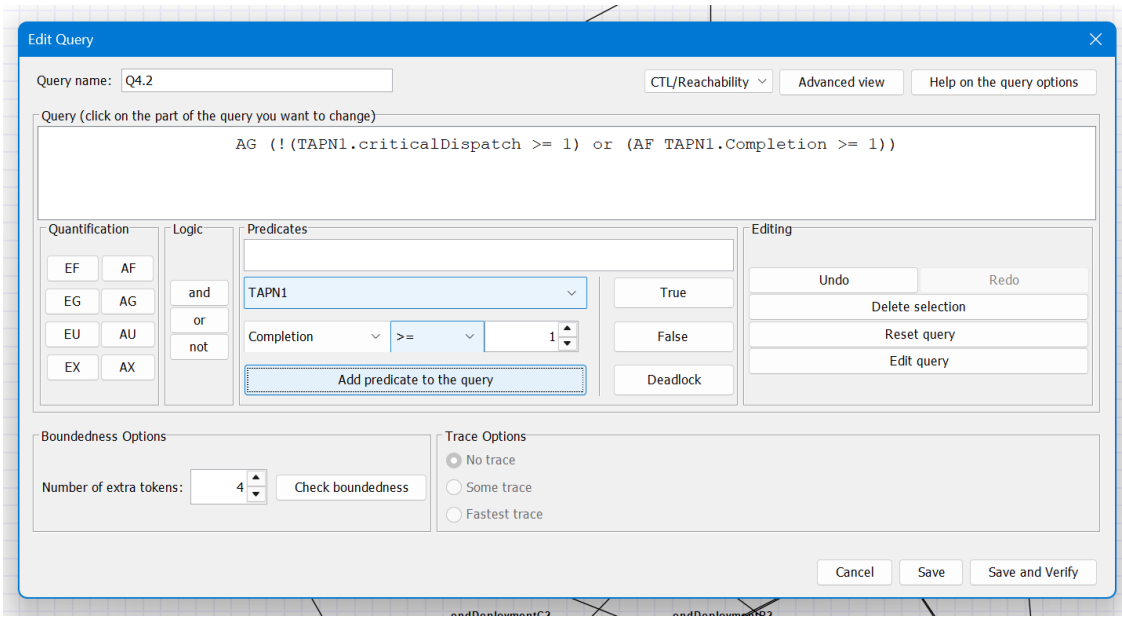
i.

$AG(\neg(criticalDispatch \geq 1) \vee AF(Completion \geq 1))$

This query states that on all execution paths, either no critical update is waiting to be dispatched, or eventually a deployment completes. This ensures that whenever a critical update is waiting, eventually a deployment completes.

ii.

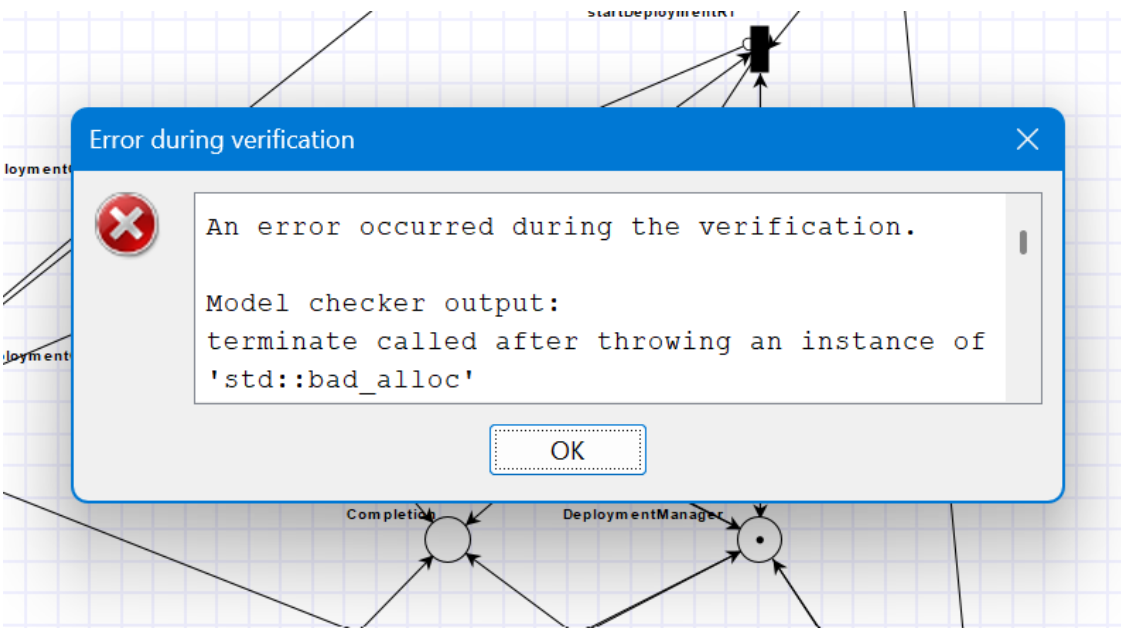
```
AG (!(criticalDispatch >= 1) or AF (Completion >= 1))
```



iii.

TAPAAL failed to evaluate this query. I tried a few times but consistently ran out of memory. I also tried reducing the number of extra tokens to zero but it still failed with the following error repeatedly after some time.

Since the query could not be completed, we cannot conclude whether critical updates are guaranteed to eventually complete. However, given that there are no blocking conditions for critical updates in the model, it is expected that they do eventually progress to completion.



4.3 Regular Update Starvation

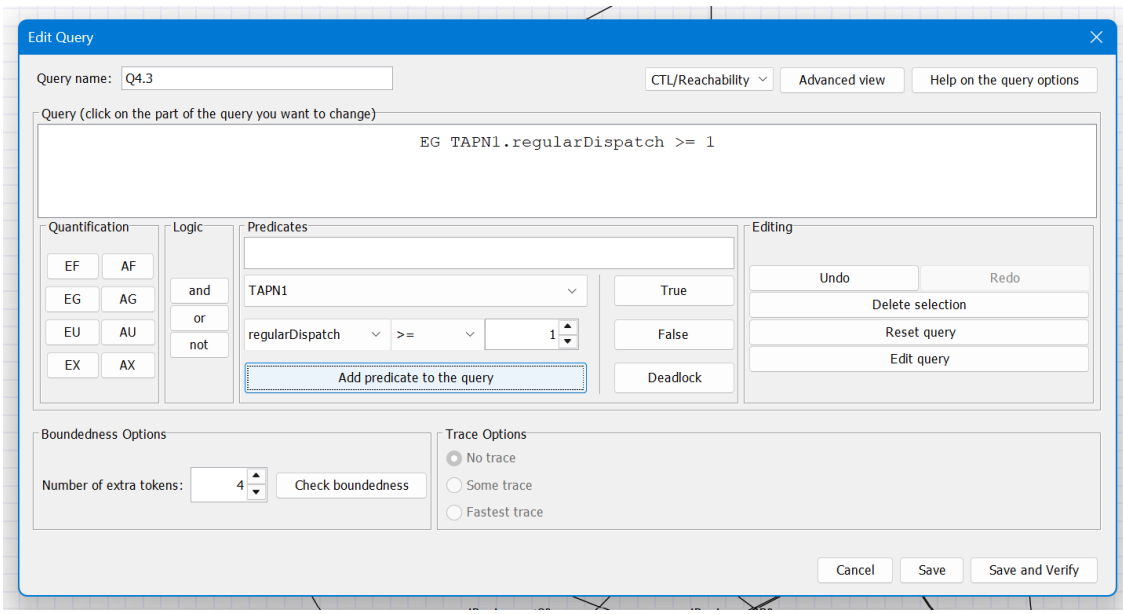
i.

$EG(regularDispatch \geq 1)$

This query checks whether there exists an execution path along which a regular update remains waiting in the dispatch place indefinitely, indicating starvation.

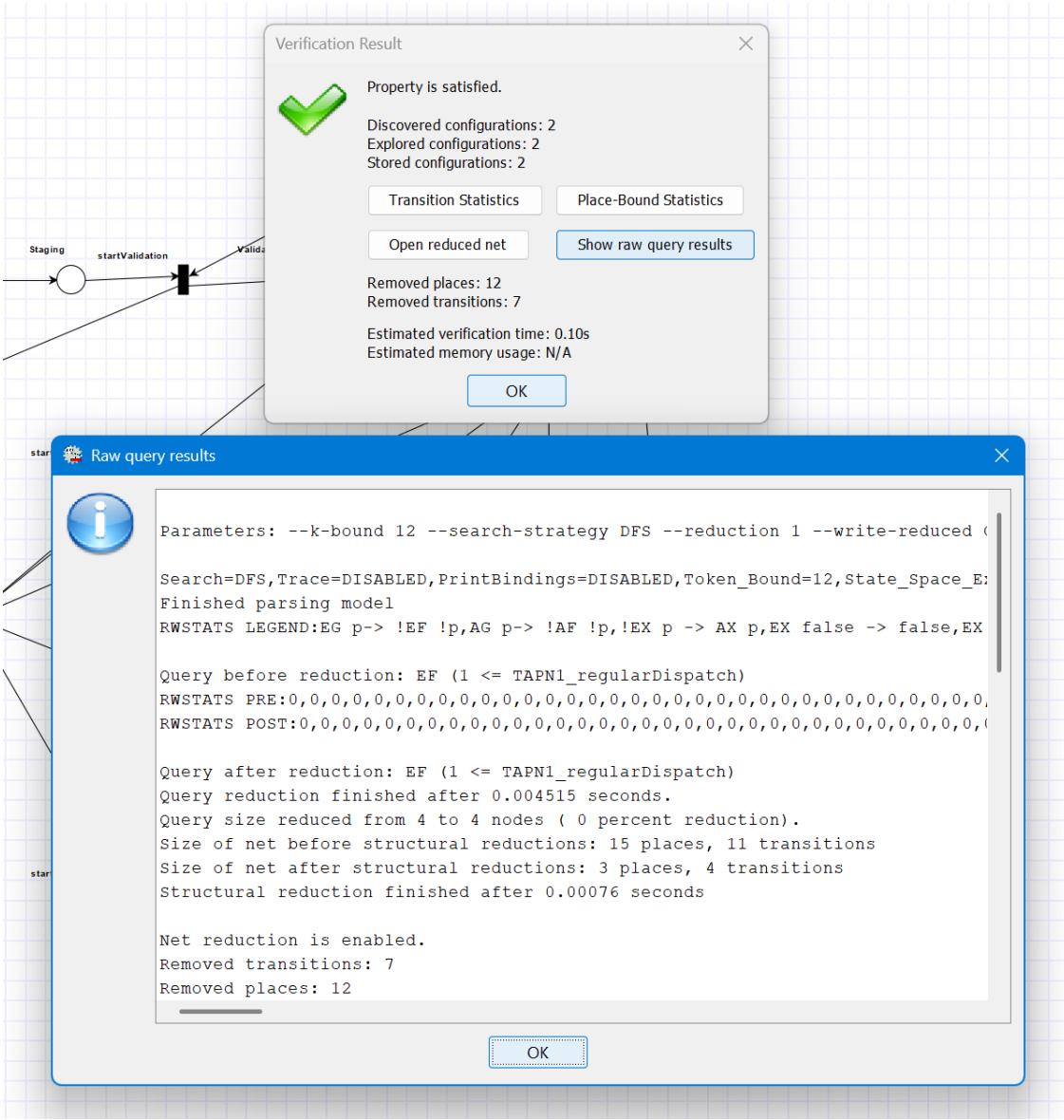
ii.

```
EG (regularDispatch >= 1)
```



iii.

TAPAAL evaluates this query as TRUE. This means that there exists an execution in which a regular update remains waiting forever. Therefore, the model does exhibit starvation of regular updates, and all regular updates will not necessarily be eventually processed.



5 Reflection

1. Verification could be useful in my future industry work when software has strong correctness requirements, especially in concurrent or safety-critical systems where errors such as deadlocks, starvation, or invalid states may be hard to detect through testing alone. This assignment showed me that verification and validation serve different purposes: verification aims to prove that specified properties hold over all possible executions, while validation checks selected executions and scenarios. In practice, I would primarily rely on validation (testing) in industry due to its scalability, relevance to end-users and business requirements, and lower cost; however, I would use verification for safety-critical or highly concurrent systems where guarantees are required. Verification is more rigorous, but also more complex and resource-intensive.

2. Tools such as TAPAAL are important for applying formal verification methods in practice. Without tool support, reasoning about all possible system states would be infeasible. The availability of these tools directly reduces the mathematical and computational burden on engineers. A good tool can make verification more accessible and practical, resulting in wider adoption of formal verification techniques in industry.

3. Verification provides strong guarantees about system behavior by proving properties over all possible executions, while validation only increases confidence by sampling executions. This assignment emphasized how verification can detect issues (such as deadlocks or starvation) that may be difficult to observe through testing alone. The link between verification and validation is that they are complementary approaches: verification provides formal guarantees using a model, while validation checks the system's behavior in practice. Together, they provide stronger assurance of correctness than either approach alone.

4. The most challenging part of this assignment was formulating correct CTL queries to capture the intended system properties. Translating informal requirements into formal logic required careful reasoning about system states and execution paths. Also, wrangling with 4.2 was a bit of a challenge, as it would repeatedly take several minutes to run and eventually error.